TÌM KIẾM VÀ PHÂN TRANG

MSc. Ngô Ngọc Đăng Khoa

TÌM KIẾM VÀ PHÂN TRANG

NỘI DUNG BUỔI HỌC

01 Pagination và các kiểu hiển thị

02 Sort/filter trên dữ liệu đã tải

03 Basic và advanced search tại server

04 Fuzzy search và tài liệu tự học full-text

05 Bài tập

CÂU HỎI XUYÊN SUỐT

> Khi danh sách chỉ hiện một phần dữ liệu, thao tác nào tác động tới phần đang có, thao tác nào tác động tới toàn bộ tập kết quả?

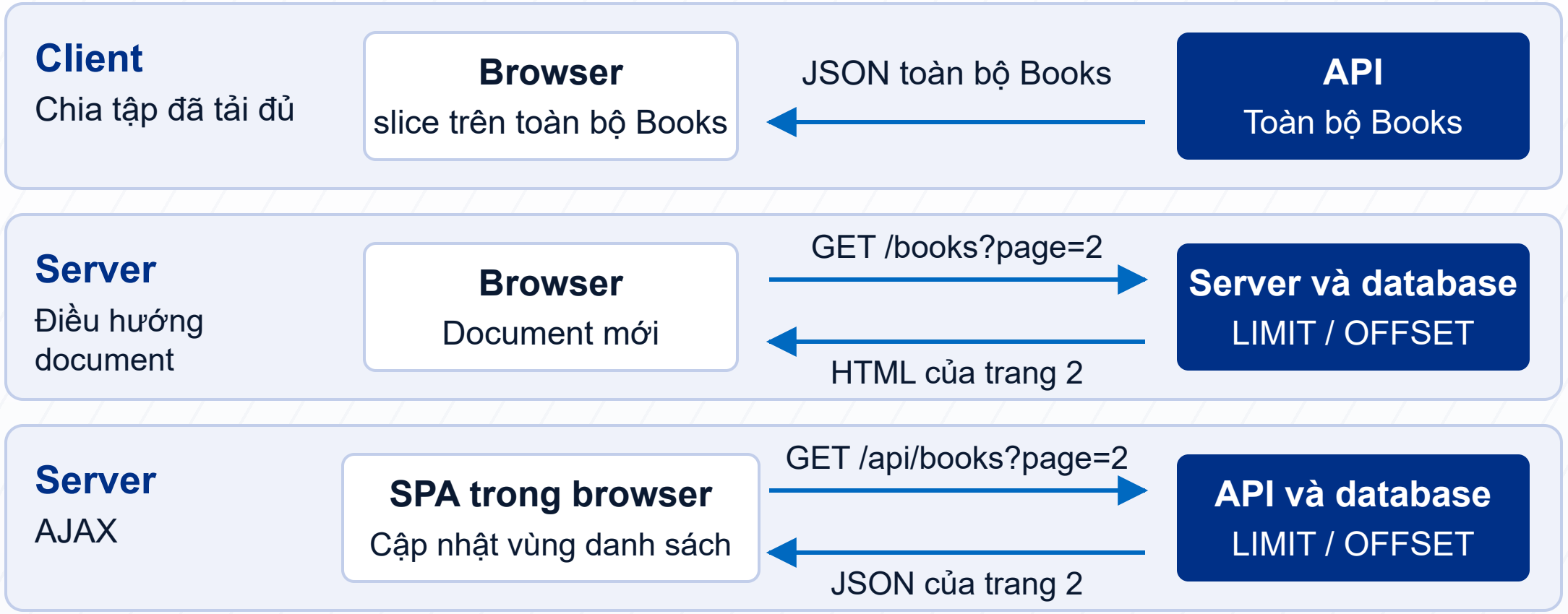
MỤC TIÊU

- Phân biệt nơi phân trang, cách cập nhật và kiểu UI
- Sort/filter đúng phạm vi dữ liệu browser đã nhận
- Chọn cơ chế search và cấu hình PostgreSQL tương ứng
- Đọc query, URL và response để nghiệm thu mã của agent

PAGINATION

QUYẾT ĐỊNH

Thành phần chia dữ liệu	Client	**Server**
Cách cập nhật Thông tin browser nhận?	HTML mới	**AJAX**
Kiểu tương tác Người dùng yêu cầu thêm nội dung?	**Số trang**	Load More / Infinite Scroll


Client
Chia tập đã tải đủ
Browser
slice trên toàn bộ Books
JSON toàn bộ Books
API
Toàn bộ Books
Server
Điều hướng document
Browser
Document mới
GET /books?page=2
HTML của trang 2
Server và database
LIMIT / OFFSET
Server
AJAX
SPA trong browser
Cập nhật vùng danh sách
GET /api/books?page=2
JSON của trang 2
API và database
LIMIT / OFFSET

OFFSET

```sql
-- Ví dụ giả định: page = 3, pageSize = 10
SELECT id, title, description, category, price, available
FROM books
ORDER BY id ASC
LIMIT 10 OFFSET 20;
```

- `offset = (page - 1) * pageSize`
- `ORDER BY id ASC` cố định thứ tự của danh sách
- Thêm hoặc xóa dữ liệu vẫn có thể làm lệch offset

RESPONSE CÓ CHỨA PAGINATION

```
{
  "ok": true,
  "data": {
    "items": [],
    "total": 23, "page": 4, "pageSize": 10
  },
  "error": null
}
```

- Ví dụ giả định: trang 4 hợp lệ nhưng không có items
- `totalPages = Math.ceil(total / pageSize)` cho kết quả `3`
- `total` không phải số phần tử của trang hiện tại

LƯU Ý

Ví dụ: `page=1`, `pageSize=10`, tối đa `100`

01

Hợp lệ: số nguyên dương, trang vượt phạm vi trả `items: []`

02

Không hợp lệ: status `200`, `ok: false`, mã `VALIDATION_FAILED`

03

Lỗi kỹ thuật: route sai hoặc lỗi hệ thống, dùng `404/500`

04

BA UI DÙNG CÙNG API PHÂN TRANG

Số trang

Trang 2 đang hiển thị

Books 4, 5, 6

1 | 2 | 3

Thay danh sách

Chọn trang cần xem

Load More

Books 1, 2, 3

Books 4, 5, 6

Tải thêm

Nối tiếp

Người dùng chủ động

Infinite Scroll

Books 1, 2, 3

Books 4, 5, 6

Sentinel được theo dõi

Nối tiếp

Sentinel xuất hiện:
kích hoạt quá trình tải

LOAD MORE

- Giữ `nextPage`, danh sách tích lũy, `pending` và `end`
- Chặn tải tiếp khi `pending` hoặc `end` đang đúng
- Thành công: nối `data.items`, rồi tăng `nextPage`
- Hết dữ liệu: `page * pageSize ⩾ total` hoặc items rỗng

INFINITE SCROLL

```
const observer = new IntersectionObserver(entries ⇒ {
  if (entries.some(entry ⇒ entry.isIntersecting)) {
    loadNext();
  }
}, { rootMargin: "200px 0px" });

observer.observe(sentinel);
// Khi rời trang: observer.disconnect();
```

- `sentinel` đặt sau danh sách, không phải từng cuốn sách
- `loadNext()` vẫn kiểm tra `pending` và `end`

MỞ LẠI ĐÚNG KẾT QUẢ QUA URL

```
/books?q=web&page=2&pageSize=10
```

TRÊN URL	GIAO DIỆN CẦN KHÔI PHỤC
`q=web`	Ô tìm kiếm có từ khóa `web`
`page=2`	Mở trang 2 của kết quả
`pageSize=10`	Hiển thị tối đa 10 sách mỗi trang

Copy link sang tab khác hoặc Reload vẫn mở lại các lựa chọn này.

KHI TÌM LẠI HOẶC CHUYỂN TRANG

Tìm kiếm: áp dụng từ khóa mới, bắt đầu ở trang 1 — 01

Chuyển trang: giữ từ khóa, chỉ đổi số trang trên URL — 02

Xóa điều kiện: hiện lại danh sách, bắt đầu ở trang 1 — 03

Reload, Back/Forward: đọc URL để khôi phục lựa chọn — 04

SORT & FILTER (CLIENT-SIDE)

FILTER: CHỈ GIỮ SÁCH THỎA ĐIỀU KIỆN

SÁCH	THỂ LOẠI	CÒN HÀNG
Web căn bản	Công nghệ	Có
SQL nhập môn	Công nghệ	Không
Truyện ngắn	Văn học	Có

Ví dụ giả định: chọn **Công nghệ** và **Còn hàng**, chỉ hiện **Web căn bản**.

SORT: ĐỔI THỨ TỰ, KHÔNG LOẠI BỚT SÁCH

THỨ TỰ GIÁ TĂNG DẦN	GIÁ GIẢ ĐỊNH
SQL nhập môn	80,000
Truyện ngắn	90,000
Web căn bản	120,000

CLIENT CHỈ XỬ LÝ DỮ LIỆU ĐÃ NHẬN

- Đã tải đủ danh sách: lọc và sort trên toàn bộ danh sách đó
- Mới tải một trang: chỉ lọc và sort được các sách của trang đó
- Muốn áp dụng cho toàn bộ kết quả: phải có đủ tập dữ liệu cần xử lý

SEARCH (SERVER-SIDE)

HAI CHIỀU THÔNG TIN CỦA CHỨC NĂNG SEARCH

CHIỀU THIẾT KẾ	CÁC LỰA CHỌN	CÂU HỎI CẦN TRẢ LỜI
Giao diện và tiêu chí	Basic, advanced	Người dùng nhập những gì?
Quy tắc so khớp văn bản	Literal contains, fulltext, fuzzy	Khi nào một sách được chọn?

- Basic không có nghĩa là tìm ở client; advanced không đồng nghĩa với `fulltext-search`
- Nhiều tiêu chí có thể kết hợp với một quy tắc so khớp văn bản

SERVER TÌM TRÊN TOÀN BỘ COLLECTION

- SPA gửi tiêu chí: `GET /api/books?mode=advanced&q=sql&available=false`
- Server chuẩn hóa, validate, gắn tham số vào SQL rồi tìm sách
- Trả `{ items, total, page, pageSize }` trong `data` của response
- Không có kết quả là `ok: true`, `items: []`, `total: 0`

FIXTURE BOOKS GIẢ ĐỊNH

ID	TITLE	CATEGORY	PRICE (ĐỒNG)	AVAILABLE
1	PostgreSQL guide	database	120000	true
2	SQL for beginners	database	80000	false
4	Cats and rats	fiction	60000	true
5	Running with cats	fiction	70000	true
6	100% SQL_notes	database	0	false
7	Postgres guide	database	110000	true

BASIC: TITLE CHỨA CHUỖI NHẬP

```
PREPARE basic_search(text) AS
SELECT id, title
FROM books
WHERE title ILIKE $1 ESCAPE '!'
ORDER BY id ASC;
```

- Nhập `SQL`, server bind pattern `%SQL%`; kỳ vọng id `1, 2, 6`
- Dùng parameter binding của driver, không nối input vào câu SQL

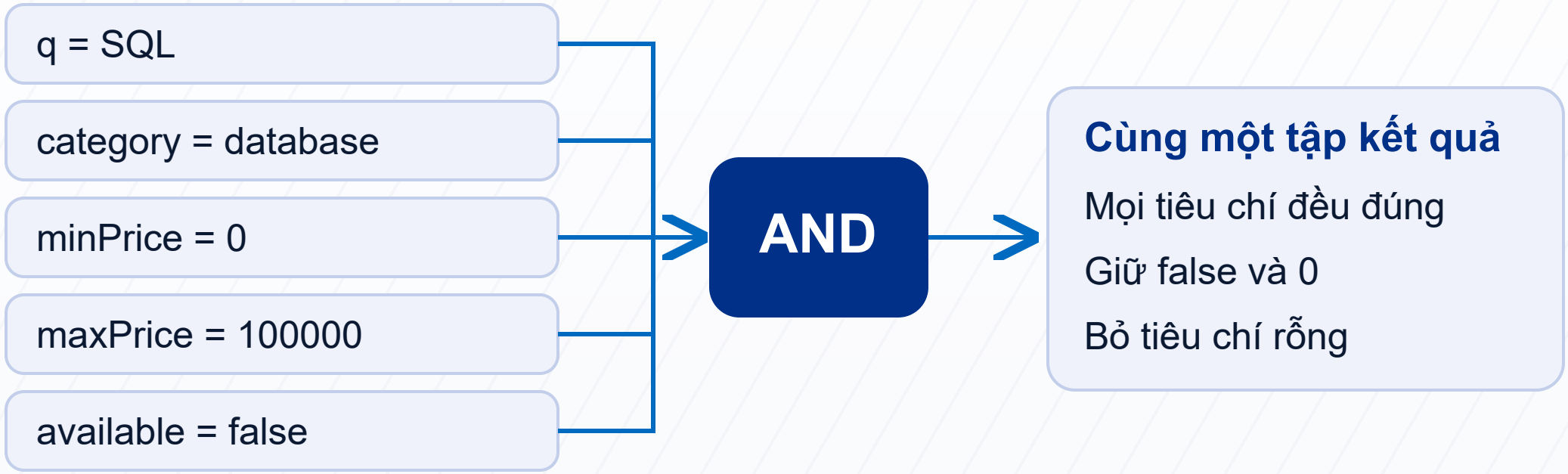
LƯU Ý: KÝ TỰ ĐẶC BIỆT VẪN LÀ DỮ LIỆU

NGƯỜI DÙNG NHẬP	PATTERN ĐƯỢC BIND	Ý NGHĨA
`100%`	`%100!%%`	Chứa đúng `100%`
`SQL_`	`%SQL!_%`	Chứa đúng `SQL_`
`!`	`%!!%`	Chứa đúng `!`

- `%` và `_` do người dùng nhập không được mở rộng phạm vi tìm kiếm
- Một chuỗi không khớp phải trả danh sách rỗng, không trả toàn bộ sách

ADVANCED: GHÉP NHIỀU TIÊU CHÍ

MINH HỌA: tiêu chí đã chuẩn hóa tại server



PHÂN BIỆT QUERY RỖNG, FALSE, 0

- Thiếu trường hoặc chuỗi trắng: chuẩn hóa thành không có tiêu chí
- `available=false`: chỉ sách không có sẵn, không được bỏ điều kiện
- `minPrice=0`: một cận giá hợp lệ, không phải giá trị rỗng
- Số sai định dạng, boolean lạ, hoặc `minPrice > maxPrice`: `VALIDATION_FAILED`

MỘT TRUY VẤN ADVANCED CÓ THAM SỐ

```
PREPARE advanced_search(text, text, numeric, numeric, boolean) AS
SELECT id, title
FROM books
WHERE ($1 IS NULL OR title ILIKE $1 ESCAPE '!')
  AND ($2 IS NULL OR category = $2)
  AND ($3 IS NULL OR price ⩾ $3)
  AND ($4 IS NULL OR price ⩽ $4)
  AND ($5 IS NULL OR available = $5)
ORDER BY id ASC;
```

SEARCH VÀ FILTER

- Basic và advanced phải cho cùng kết quả khi chỉ có cùng `q`
- `total` đếm cùng predicate mà truy vấn `items` đang dùng
- Chỉ sau khi chọn tập kết quả mới áp thứ tự và phân trang

FULL-TEXT SEARCH: TÀI LIỆU TỰ HỌC

- Supabase: Full Text Search
- MySQL Tutorial: MySQL Full-Text Search

FUZZY: CHỌN THEO ĐỘ GẦN KÝ TỰ

MINH HỌA: kỳ vọng với fixture Books giả lập



Ngưỡng quyết định giữ sách nào; similarity quyết định thứ tự sách được giữ.

BÀI TẬP

ỨNG DỤNG NHÀ HÀNG

CHUẨN BỊ

- Lưu ít nhất **1.000 món ăn trong database Supabase**
- Có script seed để tạo lại dữ liệu; giữ chức năng CRUD

PHÂN TRANG MÓN ĂN: BA CÁCH HIỂN THỊ

CÁCH HIỂN THỊ	MONG MUỐN
Số trang	Chọn trang để thay danh sách đang xem
Load More	Bấm Tải thêm để nối món vào danh sách
Infinite Scroll	Cuộn gần cuối để tự nối phần tiếp theo

NHÓM TỰ CHỌN GIẢI PHÁP PHÂN TRANG

- **Client-side:** tải đủ tập dữ liệu cần dùng rồi chia tại browser
- **Server-side:** server chọn phần dữ liệu cho mỗi yêu cầu
- **Cập nhật giao diện:** tải trang HTML mới hoặc dùng AJAX
- Giải thích lựa chọn bằng lượng dữ liệu tải và hành vi khi chuyển trang

FILTER VÀ SORT Ở CHỨC NĂNG PHÙ HỢP

- Danh sách món: lọc category, khoảng giá hoặc trạng thái còn bán
- Danh sách món hoặc quản trị: sắp xếp tên, giá tăng hoặc giảm
- Chọn vị trí áp dụng và nêu rõ tập dữ liệu bị tác động

SEARCH TẠI SERVER

MODE	YÊU CẦU
`basic`	Tìm một phần tên, không phân biệt hoa/thường
`advanced`	`q`, category, minPrice, maxPrice, available; kết hợp AND
`fulltext`	Tìm trên tên và mô tả bằng PostgreSQL full-text
`fuzzy`	Tìm tên gần giống bằng `pg_trgm`

CẤU HÌNH POSTGRESQL CHO BÀI TẬP

- Full-text: `simple`, cùng cấu hình cho document và query
- Document gồm tên và mô tả, xử lý trường có thể `NULL`
- Fuzzy: bật `pg_trgm`, công bố ngưỡng similarity `0.3`
- Full-text/fuzzy theo điểm giảm dần, thêm `id` khi điểm bằng nhau

MỞ LẠI KẾT QUẢ VÀ THAY ĐỔI ĐIỀU KIỆN

- Số trang: copy URL hoặc Reload giữ điều kiện và trang đang xem
- Tìm lại, đổi filter hoặc sort: bắt đầu từ phần đầu của kết quả mới
- Load More, Infinite Scroll: không trộn kết quả của hai lần tìm
- Công bố khi mở lại link sẽ tải từ đầu hay khôi phục phần đã xem

NGHIỆM THU BẰNG KẾT QUẢ QUAN SÁT ĐƯỢC

THAO TÁC	KẾT QUẢ CẦN CHỨNG MINH
Chọn trang, Tải thêm, cuộn tiếp	Ba kiểu UI hoạt động, không mất hoặc lặp món
Đổi filter hoặc sort	Đúng điều kiện, thứ tự và phạm vi đã công bố
Chuyển trang sau khi search	Giữ đúng bộ điều kiện, không trộn kết quả
Xóa điều kiện, Reload, mở lại link	Khôi phục danh sách hoặc lựa chọn như thiết kế

TRÌNH BÀY QUYẾT ĐỊNH CỦA NHÓM

- Chức năng nào dùng client paging, server paging hoặc AJAX?
- Filter và sort đặt ở đâu, áp dụng cho dữ liệu nào?
- Network cho thấy tải toàn bộ hay từng phần?
- Nhóm đã phát hiện lỗi nào và chứng minh sửa đúng ra sao?

BẰNG CHỨNG NỘP

- Số bản ghi trong Supabase và script seed ít nhất 1.000 món
- Demo ba kiểu hiển thị phân trang và filter/sort đã chọn
- URL search, Network và query thực hiện tại PostgreSQL
- Một lỗi đã phát hiện, cách tái hiện và bằng chứng sau khi sửa

Q & A